

Optimized Implementation of
Logic Functions:
Karnaugh Maps and Minimum
Sum-of-Product Forms

Karnaugh map

- The key to finding a minimum cost SOP or POS form is applying the combining property (14a for SOP or 14b for POS)
- The ***Karnaugh map*** (K-map) provides a systematic (and graphical) way of performing this operation
- Minterms can be combined by 14a when they differ in only one variable
 - $f(x,y,z) = xyz + xyz' = xy(z + z') = xy(1) = xy$
- The K-map illustrates this combination graphically

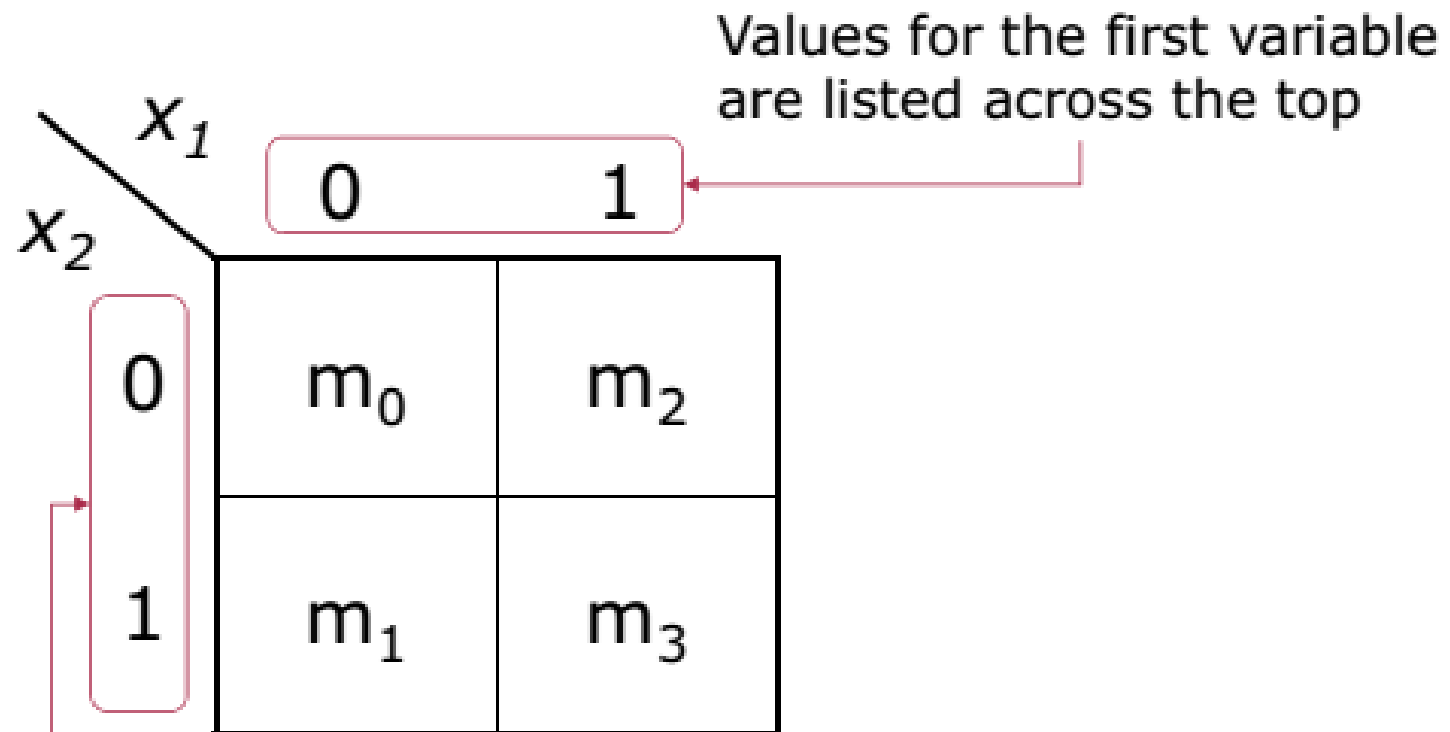
Karnaugh map

- The K-map is an alternative to a truth table for representing an expression
 - K-map consists of cells that correspond to rows of the truth table
 - Each cell corresponds to a minterm
- A two variable truth table and the corresponding K-map

x_1	x_2	f
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

		x_1	
		0	1
x_2	0	m_0	m_2
	1	m_1	m_3

Karnaugh map

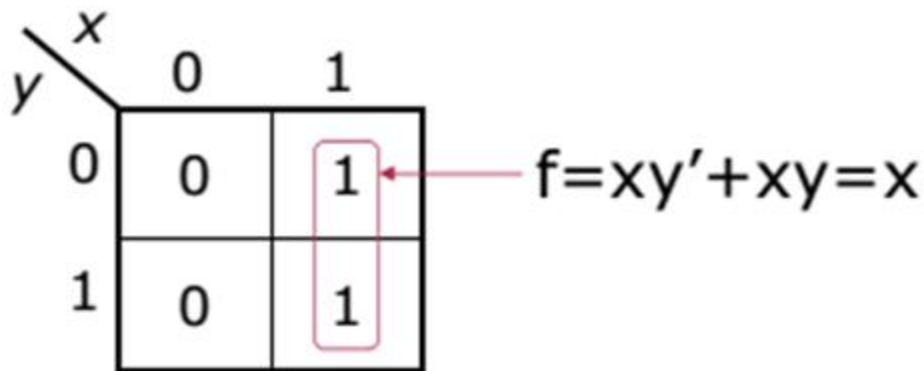


Values for the second variable are listed down the left side

Karnaugh map groupings

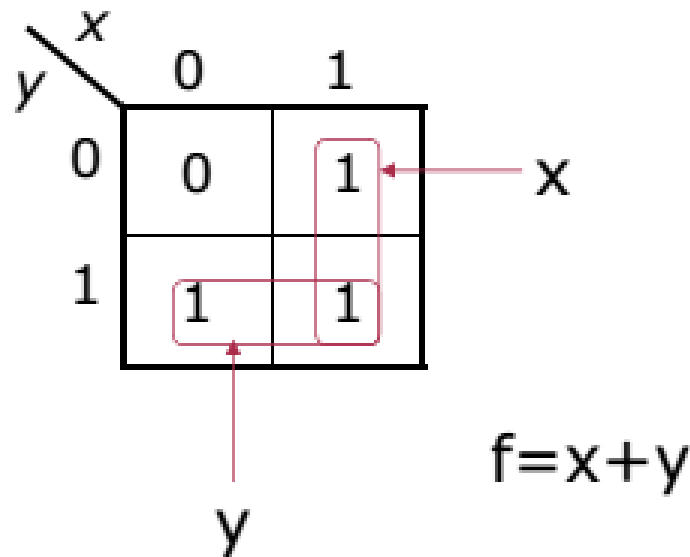
- Minterms in adjacent squares on the map can be combined since they differ in only one variable
- Indicated by looping the corresponding '1's on the map (the '1's must be adjacent)
- Looping two '1's together corresponds to eliminating a term and a variable from the output expression => $xy + xy' = x$

x	y	f
0	0	0
0	1	0
1	0	1
1	1	1



K-map groupings example

x	y	f
0	0	0
0	1	1
1	0	1
1	1	1



- Note that the bottom two cells differ in only one variable (x) and the right two cells differ in only one variable (y)

K-map groupings example

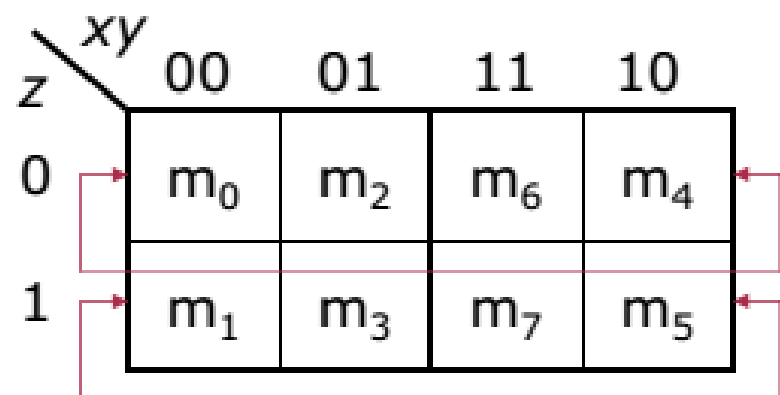
- Draw the K-map and give the minimized logic expression for the following truth table.
- Show the groupings made in the K-map

x	y	f
0	0	1
0	1	1
1	0	1
1	1	0

Three variable K-map

- A three-variable K-map is constructed by laying 2 two-variable maps side by side
- K-maps are always laid out such that adjacent squares only differ by one variable (i.e. by 1 bit in the binary expression of the minterm values)

<i>x</i>	<i>y</i>	<i>z</i>	<i>Minterm</i>
0	0	0	$m_0 = x'y'z'$
0	0	1	$m_1 = x'y'z$
0	1	0	$m_2 = x'yz'$
0	1	1	$m_3 = x'yz$
1	0	0	$m_4 = xy'z'$
1	0	1	$m_5 = xy'z$
1	1	0	$m_6 = xyz'$
1	1	1	$m_7 = xyz$



End cells are 'adjacent'

Example three-variable K-maps

$$f(x,y,z) = \sum m(0,1,2,4) \\ = x'y' + x'z' + y'z'$$

z \ xy	00	01	11	10
	0	1	1	0
0	1	1	0	1
1	1	0	0	0

$$f(x,y,z) = \sum m(0,1,2,3,4) \\ = x' + y'z'$$

z \ xy	00	01	11	10
	0	1	1	0
0	1	1	0	1
1	1	1	0	0

A grouping of four eliminates 2 variables

Guidelines for combining terms

- Can combine only adjacent `1's
- Can group only in powers of 2 (1,2,4,8, etc.)
- Try to form as large a grouping as possible
- Do not generate more groups than are necessary to "cover" all the `1's

Example groupings

$z \backslash xy$	00	01	11	10
0	1	1	1	1
1	0	0	0	0

$$f = z'$$

$z \backslash xy$	00	01	11	10
0	0	1	1	1
1	0	0	1	1

$$f = yz' + x$$

$z \backslash xy$	00	01	11	10
0	1	1	1	1
1	1	0	0	1

$$f = z' + y'$$

$z \backslash xy$	00	01	11	10
0	1	1	1	0
1	0	1	1	0

$$f = y + x'z'$$

K-map groupings example

- Draw the K-map and give the minimized logic expression for the following.
 - $f(a,b,c)=\Sigma m(1,2,3,4,5,6)$
- Show the groupings made in the K-map

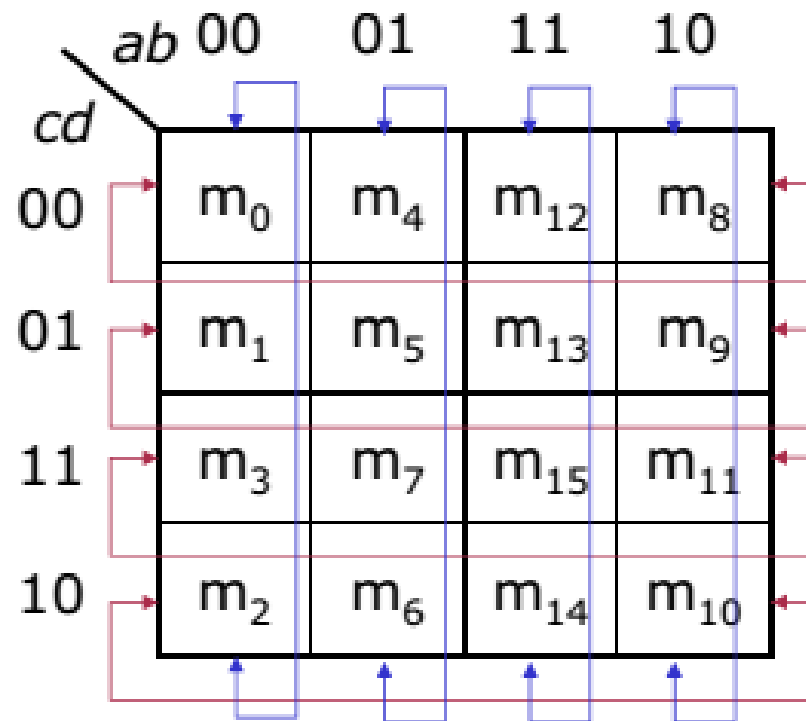
Four variable K-map

- A four-variable K-map is constructed by laying 2 three-variable maps together to create four rows
 - $f(a,b,c,d)$

$ab \backslash cd$		ab			
		00	01	11	10
cd	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Four variable K-map

- Adjacencies wrap around in the K-map



Example four-variable K-maps

$$f(a,b,c,d) = \sum m(2,3,9-11,13) \\ = ac'd + b'c$$

$cd \backslash ab$		ab			
		00	01	11	10
00	0	0	0	0	0
01	0	0	1	1	
11	1	0	0		1
10	1	0	0		1

$$f(a,b,c,d) = \sum m(3-7,9,11,12-15) \\ = b + cd + ad$$

$cd \backslash ab$		ab			
		00	01	11	10
00	0	1	1		0
01	0	1	1	1	1
11	1	1	1	1	1
10	0	1	1		0

Example groupings

ab		00	01	11	10
cd		1	1	1	1
00		1	0	0	1
01		1	0	0	1
11		1	1	1	1
10		1	1	1	1

A 4x4 Karnaugh map for the function $f(a,b,c,d) = b' + d'$. The map shows 1s in the following cells: (00,00), (01,00), (11,00), (10,00), (00,01), (10,01), (00,11), (10,11), (00,10), (01,10), (11,10), (10,10). There are two groupings: a blue grouping covering the first and fourth columns ($b=0$), and a red grouping covering the first and fourth rows ($d=0$).

$$f(a,b,c,d) = b' + d'$$

ab		00	01	11	10
cd	00	0	1	1	0
01	1	0	0	0	1
11	1	0	0	0	1
10	0	1	1	0	0

A 4x4 Karnaugh map for the function $f(a,b,c,d) = b'd + bd'$. The map shows 1s in the following cells: (01,00), (11,00), (00,01), (10,01), (00,11), (10,11), (01,10), (11,10). There are two groupings: a blue grouping covering the first and fourth columns (b=0), and a red grouping covering the second and third columns (b=1).

$$f(a,b,c,d) = b'd + bd'$$

Example groupings

		<i>ab</i>			
		00	01	11	10
<i>cd</i>	00	1	0	0	1
	01	0	1	1	0
	11	0	1	1	0
	10	1	0	0	1

$$f(a,b,c,d) = b'd' + bd$$

		<i>ab</i>			
		00	01	11	10
<i>cd</i>	00	1	1	1	0
	01	1	0	0	1
	11	1	0	0	1
	10	1	1	1	0

$$f(a,b,c,d) = b'd + bd' + a'b'$$

Terminology

- For a given term, each appearance of a variable (in true or complemented form) is called a **literal**
 - xyz' => three literals
 - $abc'd$ => four literals
- Any '1' or group of '1's that can be combined on a K-map represents an **implicant** of a function
- An implicant is a **prime implicant** if it cannot be combined with another implicant to remove a variable
- A collection of implicants that account of all valuations for which a given function is '1' is called a **cover** of that function
- **Cost** is the number of gates plus the total number of inputs to all gates in the circuit

Terminology example

ab cd		00	01	11	10
		00	01	11	10
00	1	1	0	0	
01	1	1	0	1	
11	0	1	0	1	
10	0	0	0	0	

$$f(a,b,c,d) = \sum m(0,1,4,5,7,9,11)$$

Example Implicants: all single '1's,
 $a'c'$, $a'b'c'$,
 $a'bd$, $ab'd$

Prime Implicants: $a'c'$, $a'bd$, $ab'd$, $b'c'd$

$$f(a,b,c,d)_{\min}: a'c' + a'bd + ab'd$$

Thus, a minimum SOP form contains only (but not necessarily all) prime implicants.

Prime implicants distinctions

- **Essential:** needed to form a minimum solution
- **Nonessential:** not necessarily needed to form a minimum solution

		ab			
		00	01	11	10
cd	00	0	1	0	0
	01	1	1	0	1
	11	1	0	1	1
	10	0	0	1	0

All prime implicants: $b'd$, $a'bc'$, abc
 $a'c'd$, acd

Essential primes: $b'd$, $a'bc'$, abc

Nonessential primes: $a'c'd$, acd

$f(a,b,c,d)_{\min}: b'd + a'bc' + abc$

Minimum contains all essential and possibly some nonessential primes

Prime implicants example

$cd \backslash ab$		ab			
		00	01	11	10
00	0	0	0	0	0
01	0	1	1	1	1
11	1	1	0	0	0
10	1	1	0	0	0

Essential primes: $a'c$, $ac'd$

Nonessential primes: $a'bd$, $bc'd$

One of these must be included to form a minimum solution

$$f(a,b,c,d)_{\min}: a'c + ac'd + \begin{cases} a'bd \\ bc'd \end{cases}$$

Prime implicants example

Identify all prime implicants for the given truth table. Which are essential and which are nonessential? What is a minimum SOP expression for this function?

<i>cd</i> \ <i>ab</i>					
		00	01	11	10
00	0	0	0	0	0
01	0	1	1	1	1
11	0	1	1	0	0
10	1	1	0	1	1

Minimization of POS expressions

- POS minimization using K-maps proceeds exactly as does SOP form except that groupings of '0's in the K-map are used to form POS terms.
- K-map can be constructed directly from ΠM expression for a function
- Place '0's in the K-map for every maxterm in the ΠM expression

Minimization of POS example

$$f(a,b,c) = (a+b'+c')(a'+b+c')(a'+b'+c)(a'+b'+c')$$

$$f(a,b,c) = \Pi M(3,5,6,7)$$

ab		c			
		00	01	11	10
0	1	1	0	1	
1	1	0	0	0	

$$f = (a' + b')(b' + c')(a' + c')$$

Minimization of POS example

$$f(a,b,c,d) = \prod M(0,1,4,8,10-12,14,15)$$

		<i>ab</i>			
		00	01	11	10
<i>cd</i>	00	0	0	0	0
	01	0	1	1	1
	11	1	1	0	0
	10	1	1	0	0

$$f(a,b,c,d)_{\min} = (a+b+c)(a'+c')(c+d)$$

K-map groupings example

- Draw the K-map and give the minimized POS logic expression for the following.
 - $f(a,b,c)=\Pi M(0,2,3,5-7)$
- Show the groupings made in the K-map

Incompletely specified functions

- In digital systems it often happens that some input conditions (i.e. some input valuations) can never happen
- An input combination that can never happen is referred to as a ***don't care*** condition
- As a circuit is designed, a don't care condition can be ignored (i.e. the output for that condition can be treated as 0 or 1 in the truth table)
- A function that has don't care condition(s) is said to be ***incompletely specified***

Example function with don't cares

Assume for a three variable function $f(x,y,z)$ that the input combination $xy=01$ never occurs, otherwise the function is $\Sigma m(0,1,4,5)$

x	y	z	f
0	0	0	1
0	0	1	1
0	1	0	d
0	1	1	d
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$f(x,y,z) = \Sigma m(0,1,4,5) + D(2,3)$$

Or

$$f(x,y,z) = \Pi M(6,7) \cdot D(2,3)$$

Example function with don't cares

$$f(x,y,z) = \sum m(0,1,4,5) + D(2,3)$$

$$f(x,y,z) = \prod M(6,7) \cdot D(2,3)$$

$z \backslash xy$	00	01	11	10
0	1	d	0	1
1	1	d	0	1

$$f(x,y,z) = y'$$

$z \backslash xy$	00	01	11	10
0	1	d	0	1
1	1	d	0	1

$$f(x,y,z) = y'$$

Minimum SOP form

1. Choose a minterm (a '1' in the K-map) which is not yet covered (don't consider d's).
2. Find all adjacent '1's and 'd's (check the n adjacent cells for an n-variable K-map).
3. If a single term (i.e. a single looping) covers the '1' and all adjacent '1's and 'd's then the looping forms an essential prime implicant. Loop the essential prime.
4. Repeat steps 1-3 until all essential prime implicants are located.
5. Find a minimum set of nonessential prime implicants to cover (loop) the remaining '1's. If more than 1 set is possible, choose the set with the minimum number of literals (the largest grouping).

Minimum POS form

1. Choose a maxterm (a '0' in the K-map) which is not yet covered (don't consider d's).
2. Find all adjacent '0's and 'd's (check the n adjacent cells for an n-variable K-map).
3. If a single term (i.e. a single looping) covers the '0' and all adjacent '0's and 'd's then the looping forms an essential prime implicant. Loop the essential prime.
4. Repeat steps 1-3 until all essential prime implicants are located.
5. Find a minimum set of nonessential prime implicants to cover (loop) the remaining '0's. If more than 1 set is possible, choose the set with the minimum number of literals (the largest grouping).

Multiple output circuits

- In all previous examples we have considered only single output functions
- In practice, these functions may be part of some larger circuit that has many such functions
- Circuits that implement these functions may be combined into a less costly single circuit with multiple outputs by sharing some gates needed in the implementation of the single functions

Multiple output circuit example

		ab			
cd		00	01	11	10
	00			1	1
	01		1	1	1
	11	1	1		
	10	1	1		

$$f_1(a,b,c,d) = ac' + a'c + bc'd$$

cost = 4 gates + 10 inputs

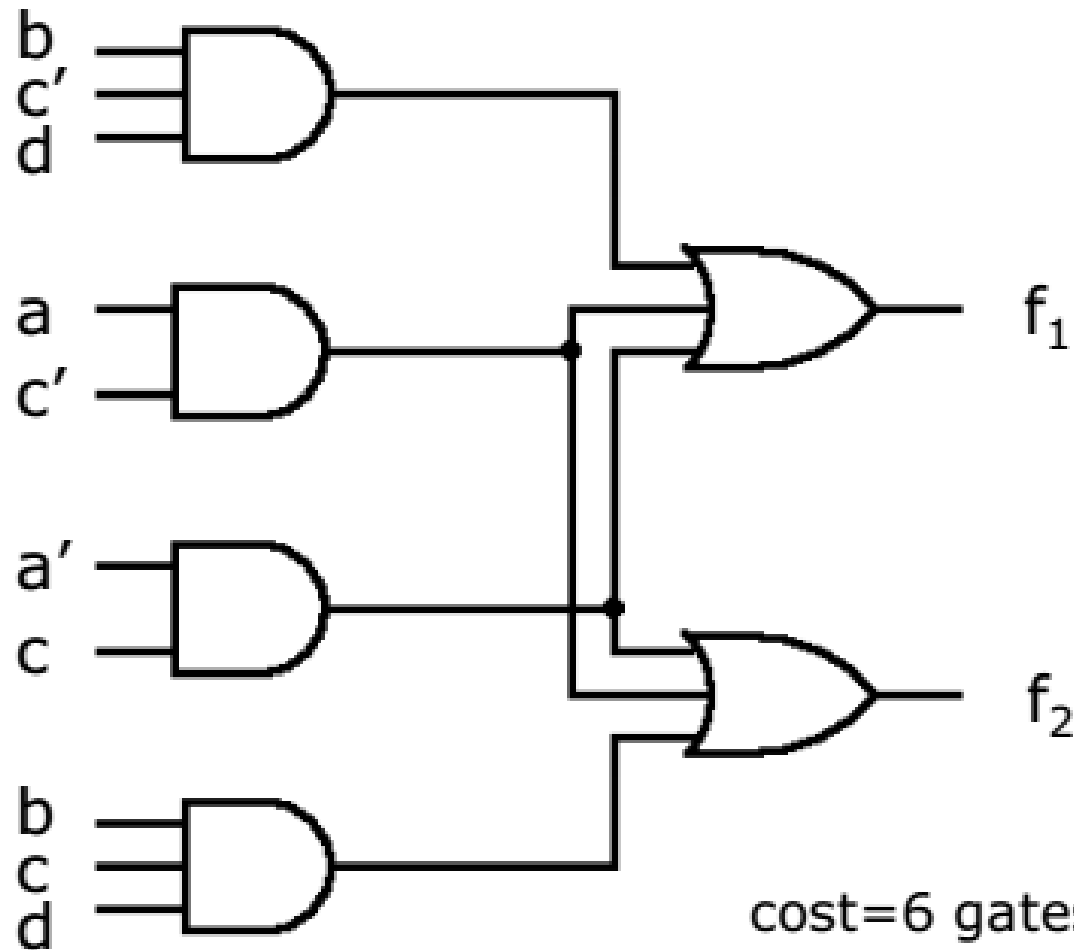
		ab			
cd		00	01	11	10
	00			1	1
	01			1	1
	11	1	1		
	10	1	1		

$$f_2(a,b,c,d) = ac' + a'c + bcd$$

cost = 4 gates + 10 inputs

NOTE: cost ignores NOT gates

Multiple output circuit example



cost=6 gates+16 inputs

Multiple output circuit example

- In this case, the minimum combined circuit was derived from the minimum circuit for each function (f_1 and f_2)
- This will not always be the case.
- Consider two functions f_3 and f_4 .

ab					
cd	00	01	11	10	
00					
01	1	1	1		
11	1	1	1		
10		1			

Optimal realization of f_3

ab					
cd	00	01	11	10	
00					
01	1		1	1	
11	1		1	1	
10		1			

Optimal realization of f_4

Multiple output circuit example

<i>cd</i> \ <i>ab</i>		00	01	11	10
		00			
	01	1	1	1	
	11	1	1	1	
	10		1		

<i>cd</i> \ <i>ab</i>		00	01	11	10
		00			
	01	1		1	1
	11	1		1	1
	10		1		

Optimal realization of f_3 and f_4 together